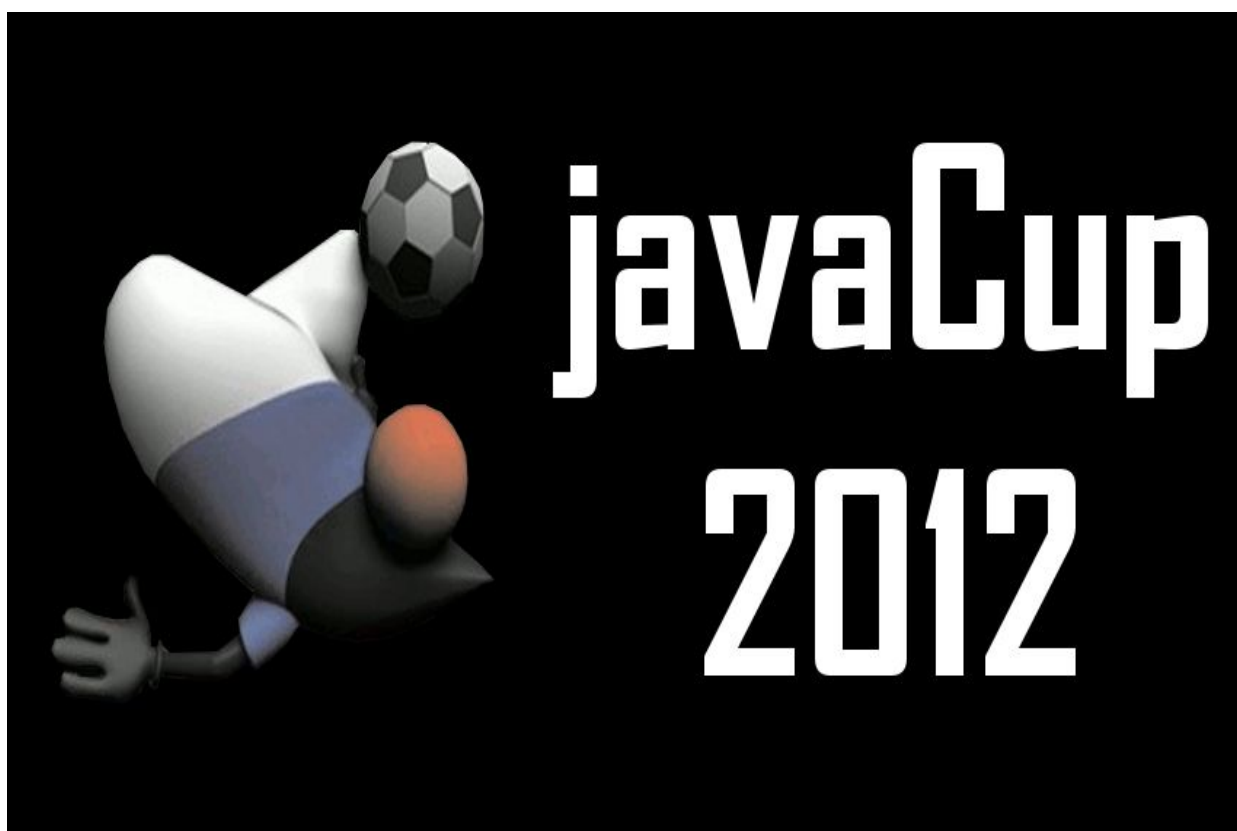




Tutorial javaCup 2012

Este tutorial explica los pasos a seguir para desarrollar una táctica y así poder participar en la javaCup.

La javaCup es un torneo de fútbol virtual donde cada participante programa en java el comportamiento de un equipo de fútbol. Luego los códigos de las tácticas enviadas se medirán en un único torneo compitiendo por los premios ofertados.

Para desarrollar una táctica solo se debe escribir una clase que implemente la interfaz `org.javahispano.javacup.model.Tactic`. Esto quiere decir que debemos implementar cada uno de los métodos definidos en dicha interfaz. Los métodos a implementar son los siguientes:

- 1) `public TacticDetail getDetail();`
- 2) `public Position[] getStartPositions(GameSituations sp);`
- 3) `public Position[] getNoStartPositions(GameSituations sp);`
- 4) `public List<Command> execute(GameSituations sp);`

El primer método denominado `getDetail` retorna un objeto `TacticDetail`. En dicho objeto se define la configuración de la táctica (nombre del equipo, el país, el nombre del entrenador, los colores y estilo de la vestimenta; y para cada jugador su nombre, número, colores, si es portero, su velocidad, su fuerza de remate y su precisión).

Para facilitar la implementación de este método se ha desarrollado un **programa asistente** que de manera gráfica permite generar el código del objeto `TacticDetail`.

El segundo y tercer método especifican la ubicación que deben tomar los jugadores después de cada gol y cuando se inicie el juego. Se usara `getStartPositions` cuando se inicie “sacando” después de haber recibido un gol, y se usara `getNoStartPositions` cuando se inicie “recibiendo” después de marcar un gol. Como el partido ocurre en un solo tiempo y por lo tanto no hay cambio de lado, al inicio ambas tácticas se ubicaran según `getNoStartPositions`, para que ambas tácticas tengan posibilidades de controlar el balón primero que la otra. Ambos métodos deberán retornar un array de objetos `Posición` de tamaño 11.

Al igual que en el primer método el **programa asistente** permitirá de manera gráfica generar los códigos necesarios para implementar los métodos `getStartPositions` y `getNoStartPositions`.

PROGRAMA ASISTENTE PARA LA CREACIÓN DE EQUIPOS

Con el fin de facilitar la creación de tácticas se ha desarrollado un programa asistente que apoyara la generación de código, la administración de alineaciones, y la simulación de remates. A continuación describiremos como se usa.

Para iniciar el asistente debes ejecutar la clase:

`JavaCupCodeGenerator`, ubicada en la raíz del código fuente.

El asistente consta de tres pestañas: “Equipo”, “Jugadores” y “Alineaciones y simulación de Remate”.

En la primera pestaña, se definen características descriptivas y de visualización del equipo. Los datos obligatorios a ingresar son el nombre del equipo, el país y el nombre del entrenador.



El botón “Al Azar” permitirá generar vestimentas al azar.

En la segunda pestaña se definen las características de los jugadores (nombre, número, ¿es portero?, color de piel, color de pelo, velocidad, potencia del remate y error del remate).

Asistente Javacup

Archivo | **Equipo** | **Jugadores** | Alineaciones y Simulacion de Remate

Nombre: N°:

Color de Piel:

Color del pelo:

Velocidad(0,33 m/iter)

Remate (2,112 m/iter)

Error (15,20 %)

Credito: 3,40

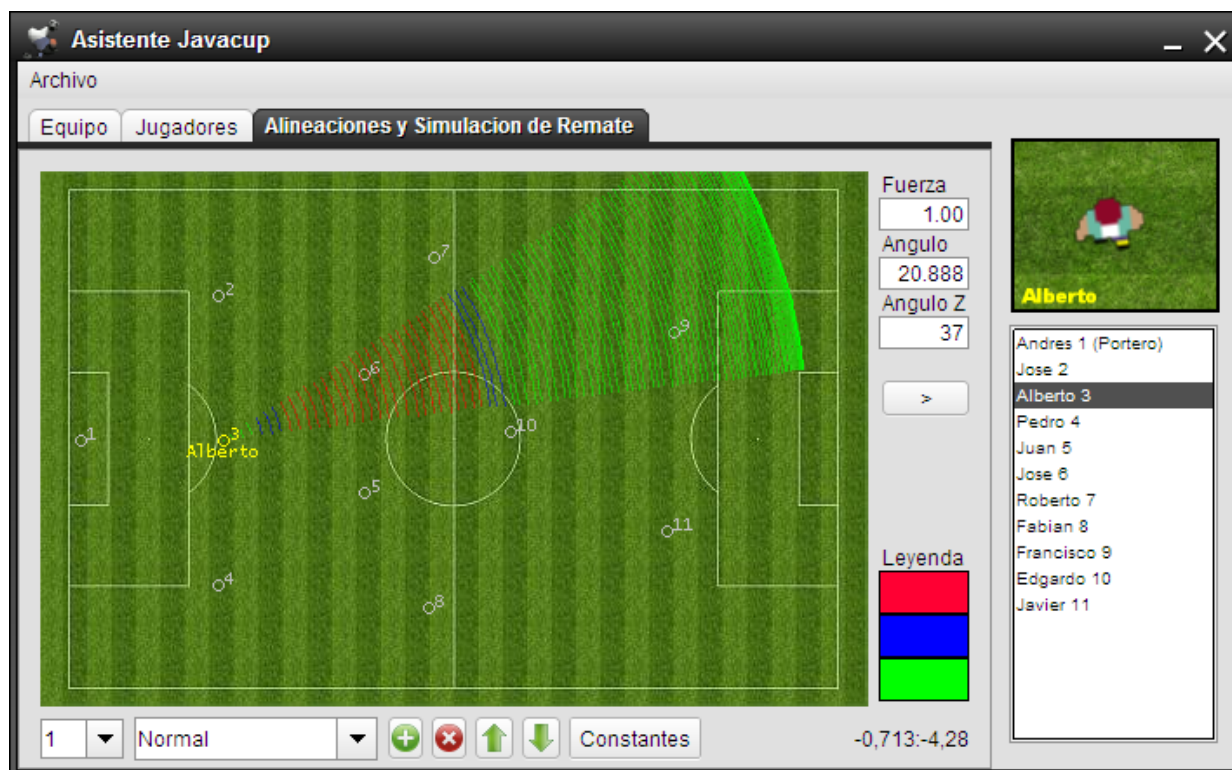
Alberto

- Andres 1 (Portero)
- Jose 2
- Alberto 3
- Pedro 4
- Juan 5
- Jose 6
- Roberto 7
- Fabian 8
- Francisco 9
- Edgardo 10
- Javier 11

Cada aptitud (velocidad, remate y error) poseen un valor mínimo y otro máximo, el valor mínimo esta a la izquierda y equivale a cero crédito usado, por otro lado, el valor máximo esta a la derecha y equivale a un crédito usado. La manera de configurar esto es distribuyendo a gusto los créditos disponibles entre los jugadores y sus aptitudes.

El error del remate se refiere a un error angular expresado en porcentaje. Por ejemplo un 15% de error significa un 15% sobre 180° , es decir $0.15 \cdot 180^\circ$, que equivale a 27° . Ahora si el remate está dirigido hacia los 90° , el ángulo final puede variar entre $90^\circ + 27^\circ/2$ y $90^\circ - 27^\circ/2$, ósea entre 76.5° y 103.5° .

Las alineaciones se configuran en la tercera pestaña. Recordemos que la interfaz *Tactic*, nos exige definir 2 alineaciones, una para cuando se inicia sacando y otra para cuando se inicia recibiendo. En el campo de juego que se muestra en la figura se pueden ubicar los jugadores y así configurar alineaciones.



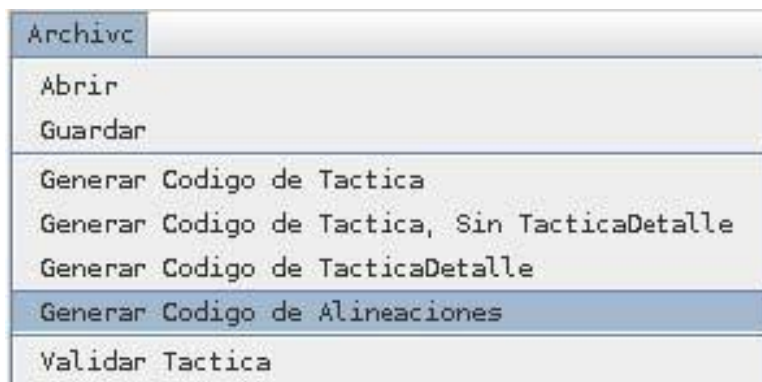
Se tienen 3 tipos de alineaciones: un tipo de alineación para ser usada durante el juego, otro tipo para cuando se inicie sacando, y otro cuando se inicie recibiendo. El tipo de alineación se configura en el segundo comboBox.

El primer comboBox mantiene la lista de alineaciones guardadas, los botones siguientes sirven para crear una nueva alineación, para eliminar la alineación seleccionada y para subir o bajar una alineación.

Los controles de la derecha (así como también, el botón derecho y el scroll del mouse) nos sirven para simular remates, indicando la fuerza [0-1], el ángulo de remate [0-360] y el ángulo vertical [0 - Constants.ANGULO_VERTICAL_MAX]. Luego con el botón ">" se simula el remate indicando las alturas del balón con colores según la leyenda de colores ubicada en la zona inferior derecha.

Por defecto se dispone de 6 alineaciones de ejemplo: 1 para iniciar sacando, 1 para iniciar recibiendo y 4 alineaciones para usarlas durante la ejecución de partidos.

El menú del asistente permite abrir, guardar, generar código y validar una táctica.



Para generar código se tienen diversas alternativas:

- 1) La primera alternativa genera el código de la clase que implementa la interfaz `Tactic` y como clase interna se incluye la clase que implementa `TacticDetail`, además se incluye el código de las alineaciones.
- 2) La segunda opción solo genera la clase que implementa `Tactic`.
- 3) La tercera opción solo genera la clase que implementa `TacticDetail`.
- 4) la cuarta alternativa solo genera el código de las alineaciones.

La opción de validación nos muestra errores que podemos estar cometiendo, como por ejemplo que no estén definidos nombres para el equipo o entrenador, o que dos jugadores tengan el mismo número, o que se ocupen mas de los créditos disponibles, etc.

EL FRAMEWORK

El framework se compone de varios paquetes, siendo `org.javahispano.javacup.model` el más importante. Esta es una lista de las clases principales ubicadas dicho paquete:

Tactic: La interfaz que deberás implementar.

TacticDetail: La interfaz que genera el asistente.

Command: Clase abstracta, que representa un comando que puede ejecutar un jugador.

CommandMoveTo: Extiende de Comando e indica a un jugador hacia dónde dirigirse.

ComandoHitBall: Extiende de Comando e indica a un jugador como rematar.

GameSituationsions: Retorna datos del estado del juego durante la ejecución de un partido.

Partido: Clase que ejecuta los partidos.

Position: coordenadas espaciales horizontales(x,y) con tipo primitivo double.

Constants: Clase donde están las constantes.

LÓGICA DE UNA TÁCTICA

```
public List<Command> execute(GameSituations sp);
```

En la implementación del método ejecutar de la interfaz `Tactic`, es donde se programa la lógica que modelará el comportamiento de tu equipo.

El método ejecutar recibe como parámetro un objeto `GameSituations`. Este objeto contiene la información del estado en que se encuentra el partido, y esta es la información que nos provee:

`double ballAltitude()` Retorna la altura del balón.

`Position ballPosition()` Retorna la posición x,y del balón.

`PlayerDetail[] rivalPlayersDetail()` Retorna la configuración (nombre, número, velocidad, remate y precisión) de los jugadores rivales.

`PlayerDetail[] myPlayersDetail()` Retorna la configuración (nombre, número, velocidad, remate y precisión) de mis jugadores.

`int myGoals()` Retorna la cantidad de goles convertidos por mi equipo.

`int rivalGoals()` Retorna la cantidad de goles convertidos por el rival.

`int iteration()` Retorna el número de iteraciones cursadas en el partido.

`int[] iterationsToKick()` Indica cuantas iteraciones deben pasar para que mis jugadores puedan volver a rematar.

`int[] rivalIterationsToKick()` Indica cuantas iteraciones deben pasar para que los jugadores rivales puedan volver a rematar.

`Position[] myPlayers()` Array de posiciones de mis jugadores.

`Position[] rivalPlayers()` Array de posiciones de rivales.

`int[] canKick()` Array de índices de jugadores míos que están en condición de rematar.

`int[] rivalCanKick()` Array de índices de jugadores rivales que están en condición de rematar.

boolean isRivalStarts() Retorna true si el rival realizara un saque, hasta que el rival no saque, mis jugadores se apartan de la posición de saque y no corre el tiempo.

boolean isStarts() Retorna true si tengo que realizar un saque, hasta que yo no saque, el rival se aparta de la posición de saque y no corre el tiempo.

double[] getTrajectory(int iteration) Retorna un array con las coordenadas x,y,z de la posición del balón en la iteración indicada.

int[] getRecoveryBall() Retorna un array donde el primer elemento es la iteración donde se puede recuperar el balón, los siguientes números corresponden a los índices de los jugadores que pueden recuperar el balón en dicha iteración, ordenados desde el más cercano al más lejano del punto de recuperación (Este método no considera el caso que los jugadores pueden obstruirse el paso).

Y Finalmente los metodos **getMyPlayerSpeed(int idx)**, **getMyPlayerPower(int idx)**, **getMyPlayerError(int idx)**, **getRivalPlayerSpeed(int idx)**, **getRivalPlayerPower(int idx)** y **getRivalPlayerError(int idx)** que retornan las aptitudes de mis jugadores y de los jugadores rivales.

Todos estos métodos nos entregan información útil para tomar decisiones, y así poder indicar qué acciones deberán realizar los jugadores. Estas acciones serán la lista de comandos que retornara el método **execute**. Sólo se tienen dos comandos disponibles; **CommandMoveTo** y **CommandHitBall**.

Algunos Ejemplos de uso:

```
new CommandMoveTo(0,new Position(0,0)); /*Se le indica al jugador de índice 0 que debe dirigirse a la posición de coordenadas 0,0. Los índices de los jugadores van de 0 hasta 10. La posición 0,0 corresponde al centro del campo de juego.*/
```

```
new CommandHitBall(6, new Position(60,30), 1, true); /*Se le indica al jugador de índice 6 que remate con dirección destino a las coordenadas (60;30), con fuerza 1, y que el remate sea por alto. La fuerza que corresponde al tercer parámetro y acepta valores double entre 0 y 1, cuando sea 1 el remate se realizara a la velocidad máxima que puede rematar el jugador de índice 6. El ultimo parámetro al ser true indica que el remate se realizara por alto. (El remate por alto se realiza con un ángulo vertical igual a Constants.ANGULO_VERTICAL */
```

```
new CommandHitBall(6, 15, 1, true); /* Otra forma de indicar el destino del remate es especificar directamente cual es ángulo de remate, en este caso es 15° */
```

```
new CommandHitBall(6, 15, 1, 25); /* Otra forma es indicar explícitamente el ángulo vertical, 25 grados en este caso. El ángulo vertical puede estar entre 0 y Constantes.ANGULO_VERTICAL_MAX. */
```

```
new CommandHitBall(6); /* Este constructor se usa para simular la acción de avanzar con el balón.*/
```

DATOS ÚTILES SOBRE LA LOGICA DE UN PARTIDO

En cada partido se ejecutaran n iteraciones, en cada iteración se enviara a las tácticas la situación del partido, y seguido a esto las tácticas retornaran la lista de los comandos que especifiquen en el método ejecutar. De acuerdo a esto se modificará el estado del partido para la siguiente iteración. En especifico se ejecutarán `Constants.ITERACIONES` iteraciones.

Un jugador sólo puede ir a una dirección y sólo puede rematar de una forma durante la iteración. Es decir, se ejecutará sólo un comando “ir a” y sólo un comando “golpear balón” para cada jugador en cada iteración.

Cuando ocurra que en una misma iteración se indique más de un `CommandMoveTo` para el mismo jugador, los primeros comandos se ignorarán y sólo se considerará el último. Análogo sucederá con el comando `CommandHitBall`.

Es importante comprender que todos los comandos actúan sobre el mismo instante de tiempo. Por esto no es válido pensar que primero ejecuto un comando “ir a” para acercarme al balón y luego que estoy cerca ejecuto golpear el balón.

Un jugador está en condiciones de rematar si se encuentra a una distancia menor o igual al balón que `Constants.DISTANCIA_CONTROL_BALON` y además que el balón se encuentre a una altura menor o igual que `Constants.ALTURA_CONTROL_BALON`. Para evitar el caso de porteros perfectos, se aplica también una probabilidad de control del balón que depende de la velocidad del balón.

Dentro de los jugadores propios y rivales que estén en condiciones de rematar se seleccionará uno al azar y sólo es ese quien rematará. Al remate se le aplicara un error angular (en el plano x,y) que dependerá del error del jugador que rematara.

Para evitar excesivos rebotes, un jugador que ya remató, sólo podrá volver a rematar en `Constants.ITERACIONES_GOLPEAR_BALON` iteraciones después.

Cada vez que el balón sale del campo de juego, deberá sacar el equipo rival del que lanzó el balón, en la posición donde salió el balón. Si un equipo lanza el balón por detrás de su propio arco se produce un tiro de esquina, que no es más que un saque normal pero en la posición de córner. En los saques de costado se reduce la potencia de remate a un 75%.

Durante las iteraciones entre que sale el balón y cuando se realiza el saque el tiempo no avanza, y los jugadores del equipo que no saca se alejarian en un radio `Constants.DISTANCIA_SAQUE` desde la posición de saque. Sin embargo, si pasa un tiempo excesivo y no se realiza el saque, se multara al equipo que no saco, permitiendo sacar al otro equipo.

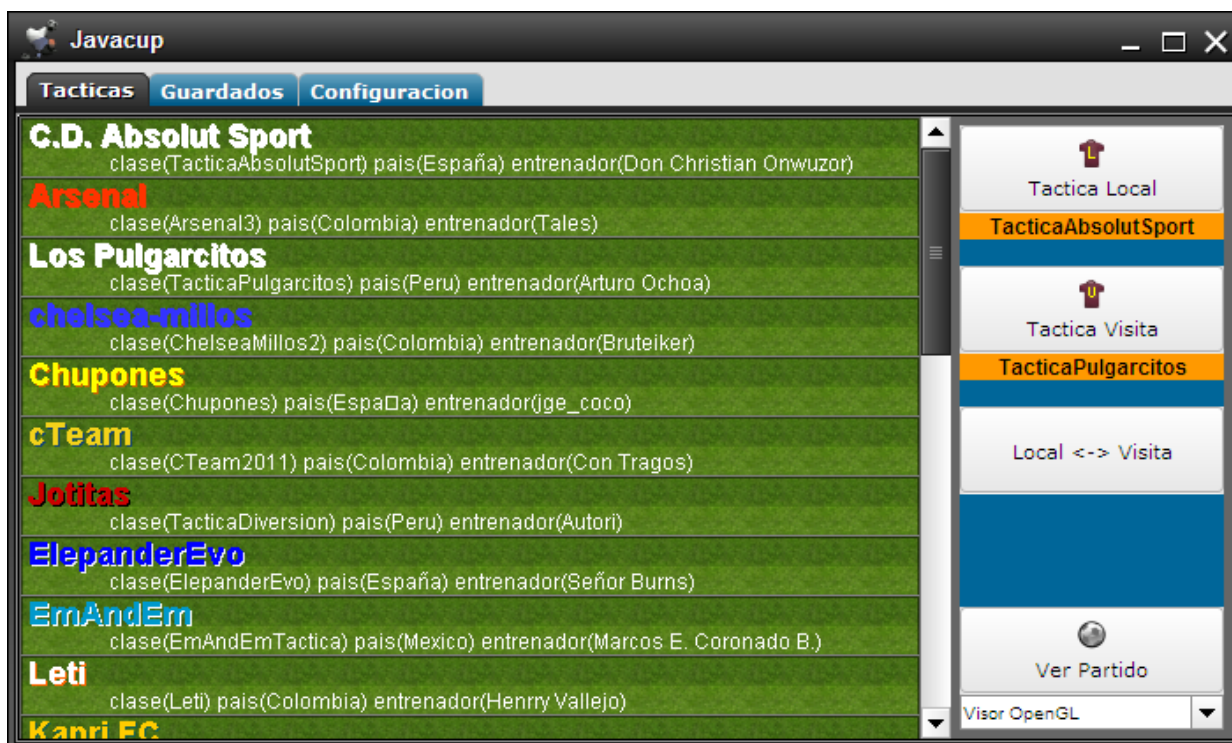
A continuación se muestra un código ejemplo de implementación del método ejecutar:

```
//Lista de comandos
LinkedList<Command> comandos = new LinkedList<Command>();

@Override
public List<Command> execute(GameSituations sp) {
    //Limpia la lista de comandos
    comandos.clear();
    //Obtiene las posiciones de tus jugadores
    Position[] jugadores = sp.myPlayers();
    for (int i = 0; i < jugadores.length; i++) {
        //Ordena a cada jugador que se ubique segun la alineacion1
        comandos.add(new CommandMoveTo(i, alineacion1[i]));
    }
    //Si no saca el rival
    if (!sp.isRivalStarts()) {
        //Obtiene los datos de recuperacion del balon
        int[] recuperadores = sp.getRecoveryBall();
        //Si existe posibilidad de recuperar el balon
        if (recuperadores.length > 1) {
            //Obtiene las coordenadas del balon en el instante donde
            //se puede recuperar el balon
            double[] posRecuperacion = sp.getTrajectory(recuperadores[0]);
            //Recorre la lista de jugadores que pueden recuperar
            for (int i = 1; i < recuperadores.length; i++) {
                //Ordena a los jugadores recuperadores que se ubique
                //en la posicion de recuperacion
                comandos.add(new CommandMoveTo(recuperadores[i],
                    new Position(posRecuperacion[0], posRecuperacion[1])));
            }
        }
    }
    //Instancia un generador aleatorio
    Random r = new Random();
    //Recorre la lista de mis jugadores que pueden rematar
    for (int i : sp.canKick()) {
        //Si el jugador es de indice 8 o 10
        if (i == 8 || i == 10) {
            //condicion aleatoria
            if (r.nextBoolean()) {
                //Ordena que debe rematar al centro del arco
                comandos.add(new CommandHitBall(i, Constants.centroArcoSup, 1, 12 + r.nextInt(6)));
            } else if (r.nextBoolean()) {
                //Ordena que debe rematar al poste derecho
                comandos.add(new CommandHitBall(i, Constants.posteDerArcoSup, 1, 12 + r.nextInt(6)));
            } else {
                //Ordena que debe rematar al poste izquierdo
                comandos.add(new CommandHitBall(i, Constants.posteIzqArcoSup, 1, 12 + r.nextInt(6)));
            }
        } else {
            //inicia contador en cero
            int count = 0;
            int jugadorDestino;
            //Repetir mientras el jugador destino sea igual al jugador que remata
            while (((jugadorDestino = r.nextInt(11)) == i
                //o mientras la coordenada y del jugador que remata
                //es mayor que la coordenada y del que recibe
                || jugadores[i].getY() > jugadores[jugadorDestino].getY())
                //Y mientras el contador es menor a 20
                && count < 20) {
                //incrementa el contador
                count++;
            }
            //Si el receptor del balon es el que remata
            if (i == jugadorDestino) {
                while ((jugadorDestino = r.nextInt(jugadores.length)) == i) {
                }
            }
            //Ordena que el jugador que puede rematar que de un pase
            //al jugador destino
            comandos.add(new CommandHitBall(i, jugadores[jugadorDestino], 1, r.nextInt(45)));
        }
    }
    //Retorna la lista de comandos
    return comandos;
}
```

VISOR DE PARTIDOS

Para usar el visor de partidos debes ejecutar la clase `JavaCup`, ubicada en la raíz del código fuente.



En la pestaña “Tácticas” aparecerán las tácticas que agregadas al código del proyecto.

Al seleccionar una táctica y presionar “Táctica Visita” asignas el equipo visita, análogo para asignar el local.

Con el botón “Local <-> Visita” se invierte el local y la visita.

En el comboBox seleccionas visualizar el partido mediante el Visor OpenGL o mediante el visor normal.

Finalmente presione “Ver Partido” para ejecutar y ver el partido.

En el Visor OpenGL:

Al presionar + y – se activa el zoom.

Al presionar F1 se activa o desactiva el estadio. Al presionar F2 se activa o desactiva el entorno. Al presionar F3 muestra los frames por segundo.

Al presiona F4 activa o desactiva los números y nombres.

En la pestaña “Guardados” están los partidos que has salvado, al visualizar uno de estos partidos puedes, avanzar, retroceder (con las flechas), marcar el una sección (con la tecla “Inicio” y “Fin”), puedes cortar la sección (con tecla “supr”) y puedes volver a guardar el partido con la tecla “S”.